

Hadoop Installation and Running Java MapReduce Program

1. Install Required Software

Install SSH and Java (OpenJDK):

```
sudo apt update  
sudo apt install ssh  
sudo apt install pdsh  
sudo apt install openjdk-11-jdk
```

Check Java installation:

```
java -version
```

Check Java folder:

```
ls /usr/lib/jvm/java-11-openjdk-amd64/
```

2. Configure Passwordless SSH

Generate SSH key:

```
ssh-keygen -t rsa -P ""
```

Add key to authorized users:

```
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

Test SSH:

```
ssh localhost
```

Exit:

exit

3. Extract Hadoop

Extract the Hadoop tar file:

```
tar -xvzf hadoop-3.*.tar.gz
```

Rename folder:

```
mv hadoop-3.* hadoop
```

Check:

```
ls
```

4. Set Environment Variables

Open `.bashrc`:

```
nano ~/.bashrc
```

Add at bottom:

```
export PDSH_RCMD_TYPE=ssh
```

```
export HADOOP_HOME=/home/pict/hadoop
```

```
export PATH=$PATH:$HADOOP_HOME/bin
```

```
export PATH=$PATH:$HADOOP_HOME/sbin
```

```
export HADOOP_MAPRED_HOME=$HADOOP_HOME
```

```
export HADOOP_COMMON_HOME=$HADOOP_HOME
```

```
export HADOOP_HDFS_HOME=$HADOOP_HOME
```

```
export YARN_HOME=$HADOOP_HOME
```

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

Reload:

```
source ~/.bashrc
```

5. Configure Hadoop Files

Go to config folder:

```
cd ~/hadoop/etc/hadoop
```

Edit core-site.xml

```
nano core-site.xml
```

Add:

```
<property>  
  <name>fs.defaultFS</name>  
  <value>hdfs://localhost:9000</value>  
</property>
```

Edit hdfs-site.xml

```
nano hdfs-site.xml
```

Add:

```
<property>  
  <name>dfs.replication</name>  
  <value>1</value>  
</property>  
  
<property>  
  <name>dfs.namenode.name.dir</name>  
  <value>file:///home/pict/hadoopdata/hdfs/namenode</value>
```

```
</property>
```

```
<property>
```

```
<name>dfs.datanode.data.dir</name>
```

```
<value>file:///home/pict/hadoopdata/hdfs/datanode</value>
```

```
</property>
```

6. Create HDFS Directories

```
mkdir -p ~/hadoopdata/hdfs/namenode
```

```
mkdir -p ~/hadoopdata/hdfs/datanode
```

7. Format NameNode

```
hdfs namenode -format
```

8. Start Hadoop

```
cd ~/hadoop
```

```
sbin/start-dfs.sh
```

Check running processes:

```
jps
```

Expected output:

```
NameNode
```

```
DataNode
```

```
SecondaryNameNode
```

```
Jps
```

Start YARN:

```
sbin/start-yarn.sh
```

Check again:

```
jps
```

Expected:

```
NameNode  
DataNode  
SecondaryNameNode  
ResourceManager  
NodeManager  
Jps
```

9. Create Input Directory in HDFS

```
hdfs dfs -mkdir /input
```

10. Put Input File in HDFS

Create a sample file:

```
nano sample.txt
```

Example text:

```
hello hadoop  
hello world  
big data hadoop
```

Upload to HDFS:

```
hdfs dfs -put sample.txt /input
```

Check file:

```
hdfs dfs -ls /input
```

11. Compile Java MapReduce Program

Assume your program is WordCount.java

Compile using Hadoop classpath:

```
javac -classpath $(hadoop classpath) -d wordcount_classes WordCount.java
```

12. Create JAR File

```
jar -cvf wordcount.jar -C wordcount_classes/ .
```

13. Run Java MapReduce Program

```
hadoop jar wordcount.jar WordCount /input /output
```

14. Check Output

```
hdfs dfs -ls /output
```

Display result:

```
hdfs dfs -cat /output/part-r-00000
```

Example output:

```
big 1  
data 1  
hadoop 2  
hello 2  
world 1
```

15. Stop Hadoop

```
stop-dfs.sh  
stop-yarn.sh
```

```
/*
 * WordCount MapReduce Program
 * -----
 * This program counts the number of occurrences of each word
 * in a given input file using the MapReduce programming model
 * provided by Apache Hadoop.
 *
 * Steps performed:
 * 1. Mapper reads input text line by line.
 * 2. Mapper splits each line into words.
 * 3. Mapper emits (word, 1).
 * 4. Hadoop groups identical keys (Shuffle & Sort phase).
 * 5. Reducer sums the values for each word.
 * 6. Final output is written to HDFS.
 */
```

```
import java.io.IOException;
import java.util.StringTokenizer;
```

```
/*
 * Hadoop configuration class used to configure the job
 */
import org.apache.hadoop.conf.Configuration;
```

```
/*
 * Represents file paths in HDFS
 */
import org.apache.hadoop.fs.Path;
```

```
/*
 * Hadoop writable datatypes
 * (used instead of normal Java String and int)
 */
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
```

```
/*
 * Core MapReduce classes
 */
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
```

```
/*
```

```

* Classes used to define input and output file paths
*/
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/*
* Main class that contains Mapper, Reducer and Driver code
*/
public class WordCount {

    /*
    * -----
    * MAPPER CLASS
    * -----
    *
    * Input Key: Object (byte offset of line in file)
    * Input Value: Text (one line of text)
    *
    * Output Key: Text (word)
    * Output Value: IntWritable (count = 1)
    */

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        // Constant value representing one occurrence of a word
        private final static IntWritable one = new IntWritable(1);

        // Hadoop Text object used to store each word
        private Text word = new Text();

        /*
        * map() method processes one line at a time
        */
        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            /*
            * Convert the line into string and split it into words
            * using StringTokenizer
            */
            StringTokenizer itr = new StringTokenizer(value.toString());

```



```

/*
 * Iterate through all tokens (words)
 */
while (itr.hasMoreTokens()) {

    // Store the current word
    word.set(itr.nextToken());

    /*
     * Emit key-value pair:
     * (word,1)
     */
    context.write(word, one);
}
}
}

```

```

/*
 * -----
 * REDUCER CLASS
 * -----
 *
 * Input Key: Text (word)
 * Input Value: List of IntWritable values
 *
 * Output Key: Text (word)
 * Output Value: IntWritable (total count)
 */

```

```

public static class IntSumReducer
    extends Reducer<Text,IntWritable,Text,IntWritable> {

    // Variable to store final sum
    private IntWritable result = new IntWritable();

    /*
     * reduce() method aggregates values for each key
     */
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context)
        throws IOException, InterruptedException {

        int sum = 0;
    }
}

```

```

    /*
    * Loop through all values associated with a word
    */
    for (IntWritable val : values) {

        // Add each value to total
        sum += val.get();
    }

    /*
    * Store final count
    */
    result.set(sum);

    /*
    * Emit final output:
    * (word,total_count)
    */
    context.write(key, result);
}
}

/*
* -----
* DRIVER / MAIN CLASS
* -----
*
* This part configures and launches the MapReduce job.
*/

public static void main(String[] args) throws Exception {

    /*
    * Create Hadoop configuration object
    */
    Configuration conf = new Configuration();

    /*
    * Create MapReduce job instance
    */
    Job job = Job.getInstance(conf, "word count");

```

```

/*
 * Specify the jar class containing the program
 */
job.setJarByClass(WordCount.class);

/*
 * Set Mapper and Reducer classes
 */
job.setMapperClass(TokenizerMapper.class);
job.setReducerClass(IntSumReducer.class);

/*
 * Define output key and value data types
 */
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);

/*
 * Set input path in HDFS
 * args[0] is the input directory
 */
FileInputFormat.addInputPath(job, new Path(args[0]));

/*
 * Set output path in HDFS
 * args[1] is the output directory
 */
FileOutputFormat.setOutputPath(job, new Path(args[1]));

/*
 * Submit job to Hadoop cluster
 * Program exits with status 0 if successful
 */
System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```